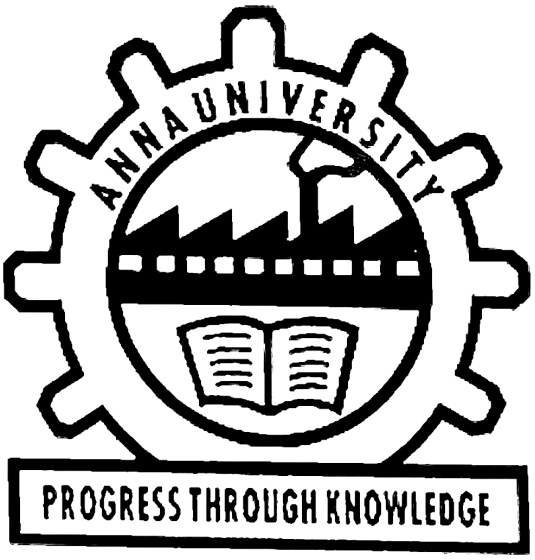
CLOUDVAULT — SECURE CLOUD STORAGE & DATA ACCESS MANAGEMENT SYSTEM

A SUMMER INTERNSHIP REPORT

Submitted by

J. PAVITHRA – 810023205035



DEPARTMENT OF INFORMATION TECHNOLOGY

ANNA UNIVERSITY ,

TIRUCHIRAPALLI – 620 024.

UNIVERSITY COLLEGE OF ENGINEERING (BIT CAMPUS),

ANNA UNIVERSITY , TIRUCHIRAPALLI – 620 024

DEPARTMENT OF INFORMATION TECHNOLOGY

BONAFIDE CERTIFICATE

This is to certify that the Summer Internship project report **"CloudVault — Secure Cloud Storage and Data Access Management System"** is the Bonafide work of **J. PAVITHRA (810023205035)** who has satisfactorily completed the course of **IT3711 – SUMMER INTERNSHIP** for **SEVENTH** semester, **B.TECH INFORMATION TECHNOLOGY** during the academic year **2025-2026**.

Faculty Incharge **Head of the Department**

Submitted for the practical examination held on _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **DLK Software Solutions, Vadapalani, Chennai**, for providing me the opportunity to undergo my Summer Internship in the domain of Web Development. This internship has been a transformative learning experience that gave me practical exposure to Python Flask, database management, and professional software development practices.

I am deeply thankful to my Company Mentor, **Mr. Ram**, for his constant guidance, technical mentorship, and encouragement throughout the internship. His insights into Flask development, SQLAlchemy ORM, secure file handling, and web application architecture were instrumental in the successful completion of the CloudVault project.

I extend my heartfelt thanks to my Faculty Incharge, **Ms. Ambika**, and the Head of the Department, Department of Information Technology, University College of Engineering (BIT Campus), Tiruchirapalli, for their academic support and for approving this internship as part of the curriculum.

I am also grateful to all the staff and fellow interns at DLK Software Solutions for creating a positive and productive learning environment during the internship period.

Finally, I express my sincere gratitude to my parents, friends, and family for their unwavering support and constant motivation throughout this journey.

ABSTRACT

The **CloudVault — Secure Cloud Storage and Data Access Management System** is a full-stack web application developed using the Python Flask framework. The system provides authenticated users with a centralized digital platform to securely upload, store, manage, and share files using unique time-limited shareable links with access control and download tracking.

The project backend is built using Python Flask, a lightweight and flexible web framework that handles URL routing, request processing, and business logic. The database layer uses SQLAlchemy ORM with SQLite for efficient management of user accounts and file metadata without requiring a separate database server. User session management is handled by Flask-Login, and password security is ensured through Werkzeug's hashing functions. The frontend is built with HTML5, CSS3, Bootstrap 5, and Vanilla JavaScript providing a fully responsive and intuitive user interface.

Core features of the system include secure user registration and login with hashed password storage, file upload with type validation (PDF, DOC, images, ZIP) and a 16MB size limit, UUID-based unique shareable link generation for every uploaded file, configurable file expiry (1, 3, 7, or 30 days) that automatically invalidates links after the set period, real-time download count tracking, a dashboard displaying aggregate statistics, and file management with delete functionality.

The project was developed during a one-month offline Summer Internship at DLK Software Solutions, Vadapalani, Chennai, from 06 June 2026 to 06 July 2026, under the guidance of Mr. Ram (Company Mentor) and Ms. Ambika (Faculty Incharge). Starting with no prior knowledge of Flask, the complete system was built within four weeks following an MVC architecture pattern.

The system was thoroughly tested across all modules including authentication, file upload, share link access, expiry validation, download tracking, search, drag-and-drop, and responsive layout. All test cases passed successfully. CloudVault demonstrates the practical application of Python Flask in delivering a secure and user-friendly web application.

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO
1	**INTRODUCTION** 1.1 Background 1.2 Objective of the Internship 1.3 Scope of the Report 1.4 Technologies Used	1
2	**COMPANY PROFILE** 2.1 About DLK Software Solutions 2.2 Internship Details 2.3 Project Undertaken	4
3	**DAILY WORK REPORT**	5
4	**BLOCK DIAGRAM** 4.1 System Architecture 4.2 Data Flow Diagram 4.3 Database Schema	7
5	**CLOUDVAULT SYSTEM** 5.1 Existing System 5.2 Proposed System 5.3 System Capabilities	10
6	**PROPOSED METHOD** 6.1 User Authentication 6.2 File Upload Module 6.3 Share Link Generation 6.4 File Expiry Control 6.5 File Download and Delete 6.6 Dashboard and File Management 6.7 Front-End Interaction	12
7	**ALGORITHM** 7.1 User Registration 7.2 User Login 7.3 File Upload 7.4 Share Link Access 7.5 File Expiry Check 7.6 File Delete 7.7 Dashboard Statistics	14
8	**CODE IMPLEMENTATION**	16
9	**SAMPLE OUTPUT**	20
10	**ANALYSIS**	23
11	**CONCLUSION**	24
12	**FUTURE WORK**	25
13	**REFERENCES**	26

1. INTRODUCTION

1.1 Background

CloudVault — Secure Cloud Storage and Data Access Management System is a Python Flask-based full-stack web application developed to provide individuals and small teams with a secure, self-hosted solution for storing and sharing files over the internet with controlled, time-limited access.

In today's digital environment, the demand for secure file storage and sharing has grown significantly across academic, professional, and personal settings. Students share project reports, professionals exchange confidential documents, and teams collaborate through shared files every day. However, most available solutions either lack fine-grained access control, are expensive, or require a complex setup.

CloudVault is designed to fill this gap by providing a lightweight, self-hosted cloud storage web application where users can upload files, generate unique shareable links with automatic expiry, track how many times each file has been downloaded, and manage their file collection through an intuitive dashboard.

Cloud storage is a model of data storage in which digital files are kept on a remote server that can be reached through a network, usually the internet, instead of on the local disk of a single computer. Its main advantages are accessibility from any location, ease of sharing, and centralized management of data. A cloud storage system must therefore answer three basic questions: who is allowed to store files, who is allowed to read them, and for how long they may be read. CloudVault answers these questions through user authentication, unique share tokens and configurable link expiry.

A shareable link is a web address that contains a long, random and practically unguessable identifier. Anyone who holds the link can download the file, but nobody can discover it without the identifier. Making such a link time-limited adds a second layer of protection, because the link stops working automatically after the period chosen by the owner. This combination of an unguessable token and an expiry date is the central idea of the CloudVault project.

1.2 Objective of the Internship

The main aim of the internship was to learn a new web technology stack from the beginning and to apply it to a complete, working project. The specific objectives were:

- Learn and apply the Python Flask web framework for building a complete web application.
- Design a relational database using the SQLAlchemy ORM with SQLite.
- Implement secure user authentication with session management using Flask-Login.
- Build file upload functionality with type validation, size restriction, and secure naming.
- Implement UUID-based unique shareable link generation with configurable expiry control.
- Develop a fully responsive multi-page web interface using Bootstrap 5 and JavaScript.
- Gain experience with Git version control and a professional software development workflow.
- Understand the MVC architecture pattern and modular code organization in Flask.
- Develop problem-solving and debugging skills through real-world project development.

1.3 Scope of the Report

This report documents the complete Summer Internship experience at DLK Software Solutions. It covers the company profile, the day-wise work log, the system architecture, module-level implementation details with code excerpts, sample outputs, test case analysis, the conclusion, and the future scope of the CloudVault project.

The report is organized as follows. Chapter 2 introduces the company and the internship details. Chapter 3 gives the daily work report. Chapter 4 explains the architecture, the data flow and the database design with block diagrams. Chapters 5 and 6 describe the existing and proposed systems and the method followed in each module. Chapter 7 presents the algorithms, Chapter 8 the code implementation, and Chapter 9 the sample output. Chapters 10 to 13 contain the analysis, conclusion, future work and references.

1.4 Technologies Used

Python and Flask. Python is a high-level, general-purpose programming language known for its readable syntax and large library ecosystem. Flask is a micro web framework written in Python. It is called a micro framework because its core is small and it does not force a particular database, form library or project layout on the developer. Flask is built on top of Werkzeug, which implements the WSGI (Web Server Gateway Interface) standard that connects a Python application to a web server, and on the Jinja2 template engine, which generates HTML pages from templates. A Flask application is created by mapping URL patterns to Python functions called routes; each route receives a request and returns a response.

SQLAlchemy and SQLite. SQLAlchemy is an Object Relational Mapper (ORM). An ORM lets the programmer describe database tables as Python classes and rows as objects, so that data can be stored and queried without writing raw SQL for every operation. SQLite is a lightweight, file-based relational database that needs no separate server process, which makes it ideal for development, learning and small deployments. Flask-SQLAlchemy connects the two to the Flask application and manages the database session.

Flask-Login and password hashing. Flask-Login manages user sessions. After a successful login it stores the identity of the user in a signed session cookie, and the @login_required decorator then restricts access to protected routes. Passwords are never stored as plain text. Werkzeug's generate_password_hash() converts a password into a salted hash using a one-way algorithm (PBKDF2-SHA256), and check_password_hash() compares a submitted password with the stored hash. Even if the database is exposed, the original passwords cannot be read directly.

UUID. A Universally Unique Identifier (UUID) is a 128-bit value that is generated randomly so that the chance of two identical values is negligible. CloudVault uses version 4 UUIDs for two purposes: as a prefix that makes each stored file name unique, and as the share token that forms the shareable link.

HTML, CSS, Bootstrap and JavaScript. HTML defines the structure of each page and CSS controls its appearance. Bootstrap 5 is a front-end framework that provides a responsive grid, ready-made components such as cards, forms, buttons and alerts, and utility classes, so that pages adapt automatically to mobile, tablet and desktop screens. JavaScript adds behavior in the browser, such as drag-and-drop upload, live search, clipboard copy and toast notifications, without reloading the page.

Git and VS Code. Git is a distributed version control system used to record the history of the source code, and Visual Studio Code is the editor used for development. Both are standard tools in professional software development, and their use during the internship gave practice in maintaining a clean and traceable code base.

2. COMPANY PROFILE

2.1 About DLK Software Solutions

DLK Software Solutions is a well-established IT training and software development organization founded in 2012, headquartered at Vadapalani, Chennai, Tamil Nadu. The organization bridges the gap between academic education and industry requirements by providing structured, hands-on training and internship programmes to engineering students and professionals across Tamil Nadu.

DLK Software Solutions follows the philosophy of **Learn – Practice – Build – Get Placed**. The organization offers training in Web Development, Mobile App Development, Data Science, Machine Learning, and Embedded Systems. Students work on real-world projects during their internship, gaining practical skills that are directly applicable in the industry.

The internship at DLK Software Solutions was conducted in offline mode at the Vadapalani centre. Interns were first introduced to the company and its working practices, then guided through the fundamentals of the technology assigned to them, and finally asked to build a complete project. This sequence of learning, practising and building allowed a beginner to move from basic concepts to a working application within one month.

2.2 Internship Details

The details of the company and of the internship undergone by the author are given below.

- **Company Name:** DLK Software Solutions
- **Established:** 2012
- **Location:** Raahat Plaza, Ground Floor, Opp. Vijaya Hospital, Vadapalani, Chennai – 600 026, Tamil Nadu
- **Phone / WhatsApp:** 77081 50152
- **Website:** www.dlksoftwaresolutions.co.in
- **Email:** info@dlksoftwaresolutions.co.in
- **Training Domain:** IT Training and Software Development
- **Internship Domain:** Web Development

- **Internship Mode:** Offline
- **Duration:** 06 June 2026 – 06 July 2026 (one month)
- **Company Mentor:** Mr. Ram
- **Faculty Incharge:** Ms. Ambika, Department of Information Technology, University College of Engineering (BIT Campus), Tiruchirappalli
- **Academic Year:** 2025 – 2026

2.3 Project Undertaken

As part of the internship, the author developed **CloudVault — Secure Cloud Storage and Data Access Management System** using Python Flask. The project involved designing a responsive interface with HTML, CSS, Bootstrap and JavaScript, developing backend services with Flask, integrating the SQLite database through SQLAlchemy, implementing authentication with Flask-Login, and testing the complete application.

The internship gave the author valuable practical knowledge of web development and a clear understanding of the Software Development Life Cycle (SDLC), from requirement analysis and design to implementation, testing and documentation. It improved programming skills, database handling, problem-solving ability and the understanding of how the front end and back end of a web application work together.

3. DAILY WORK REPORT

The internship was carried out over thirty days, from 06 June 2026 to 06 July 2026. The work of each day is described below. The first two weeks were spent on learning Flask and the database layer, the third week on building the core modules, and the last two weeks on the interface, testing and documentation.

Day 1 (06 June): Attended the induction session, was introduced to the company and its training approach, and set up the development environment by installing Python, VS Code and Git.

Day 2 (07 June): Learned what a web framework is and how Flask fits into web development, and created the first Hello World Flask application to confirm that the setup works.

Day 3 (08 June): Studied URL routing and route parameters, and practised redirect(), url_for() and the HTTP GET and POST methods.

Day 4 (09 June): Learned Jinja2 templating, including template inheritance with base.html, blocks, loops, conditionals and flash messages.

Day 5 (10 June): Practised HTML form handling, extracted submitted data with request.form and studied basic form validation techniques.

Day 6 (11 June): Installed Flask-SQLAlchemy, configured the SQLite database and initialised the db object.

Day 7 (12 June): Designed the User model with id, username, email and password columns using the SQLAlchemy ORM.

Day 8 (13 June): Designed the File model with filename, share_token, expiry_date and download_count columns and linked it to the User model.

Day 9 (14 June): Built the user registration route using generate_password_hash() and added validation for duplicate email addresses.

Day 10 (15 June): Built the login route using check_password_hash() and set up Flask-Login sessions with login_user().

Day 11 (16 June): Configured LoginManager with the user_loader callback and protected the private routes with the @login_required decorator.

Day 12 (17 June): Created config.py containing SECRET_KEY, UPLOAD_FOLDER, MAX_CONTENT_LENGTH and ALLOWED_EXTENSIONS.

Day 13 (18 June): Implemented file upload with secure_filename(), UUID prefix generation and the file save logic.

Day 14 (19 June): Stored the file name, size, type, share_token and expiry_date in the SQLite database through SQLAlchemy.

Day 15 (20 June): Generated a UUID-based share_token for every file and designed the shareable URL in the form /share/<token>.

Day 16 (21 June): Implemented the expiry logic, where expiry_date is the upload time plus the chosen number of days, together with the expiry validation on the share route.

Day 17 (22 June): Used send_from_directory() with the download_name parameter to serve files and incremented the download_count on every download.

Day 18 (23 June): Completed the /share/<token> route with the expiry check so that files can be accessed publicly without logging in.

Day 19 (24 June): Implemented deletion using os.remove() for the physical file and session.delete() for the database record, with a confirmation prompt.

Day 20 (25 June): Designed the Bootstrap 5 dashboard with statistic cards for files, downloads and active links, and a list of recent files.

Day 21 (26 June): Built the drag-and-drop upload area with a JavaScript file name preview and the expiry dropdown for 1, 3, 7 and 30 days.

Day 22 (27 June): Created files.html with file type badges, Active and Expired status badges and the download count display.

Day 23 (28 June): Added real-time file search in vanilla JavaScript so that the list filters without reloading the page.

Day 24 (29 June): Integrated the Clipboard API for the Copy Link button along with a toast notification.

Day 25 (30 June): Designed index.html with the hero section, and the register.html and login.html pages with Bootstrap card layouts.

Day 26 (01 July): Wrote style.css with the purple gradient theme (#667eea to #764ba2), hover effects and responsive breakpoints.

Day 27 (02 July): Completed main.js with drag-and-drop upload, auto-hiding alerts, file preview and toast notifications.

Day 28 (03 July): Carried out end-to-end testing of all routes, tested edge cases and checked the application in different browsers.

Day 29 (04 July): Fixed bugs in expired link handling, file type edge cases and session persistence.

Day 30 (05 – 06 July): Prepared the README.md, added code comments, documented the project structure and completed the final submission.

4. BLOCK DIAGRAM

4.1 System Architecture

CloudVault is designed using the **MVC (Model-View-Controller)** architectural pattern, one of the most widely used patterns in web development. This architecture separates the application into three distinct parts, which makes the code base modular, maintainable and easy to test. The same separation can be seen as a three-tier structure made up of the presentation layer, the business logic layer and the data layer, as shown in Figure 4.1.

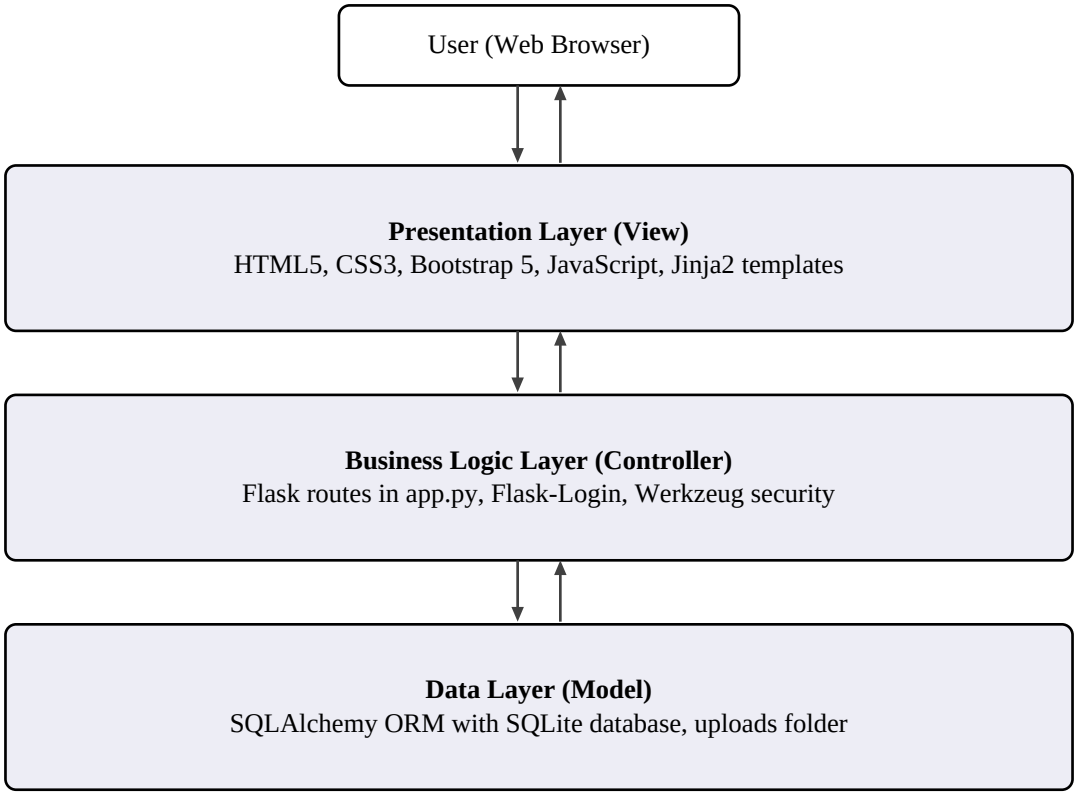


Figure 4.1 Three-tier architecture of CloudVault

The **Model** consists of the User and File classes defined in models.py with SQLAlchemy. They describe the structure of the data and perform all database operations. The **View** consists of the HTML templates written with Jinja2 and styled with Bootstrap 5; they render the user interface with dynamic data supplied by the controller. The **Controller** is app.py, whose routes receive the requests, apply the business logic, call the models and select the template that is returned to the browser.

Two supporting parts complete the architecture. The configuration module, config.py, holds application settings such as the secret key, the upload folder, the maximum file size and the allowed extensions. The static folder holds the CSS and JavaScript files, and the uploads folder on the server's file system stores the physical files that users upload.

4.2 Data Flow Diagram

Figure 4.2 shows the flow of data in CloudVault. The owner logs in, uploads a file and sets an expiry period. The backend verifies the session, validates the file type and size, saves the file under a unique name and records its details, share token and expiry date in the database. The share link is then shown to the owner, who can copy it and send it to anyone.

The upload request itself passes through several components in a fixed order. The browser sends the file in a POST request to the /upload route. Flask-Login checks the session and loads the current user. Werkzeug's secure_filename() cleans the file name, and the UUID module creates a unique prefix and a unique share token. The file is written to the uploads folder, the SQLAlchemy ORM inserts the file record into SQLite, and finally Flask redirects the user to the dashboard, where the Jinja2 template displays the updated file list and statistics.

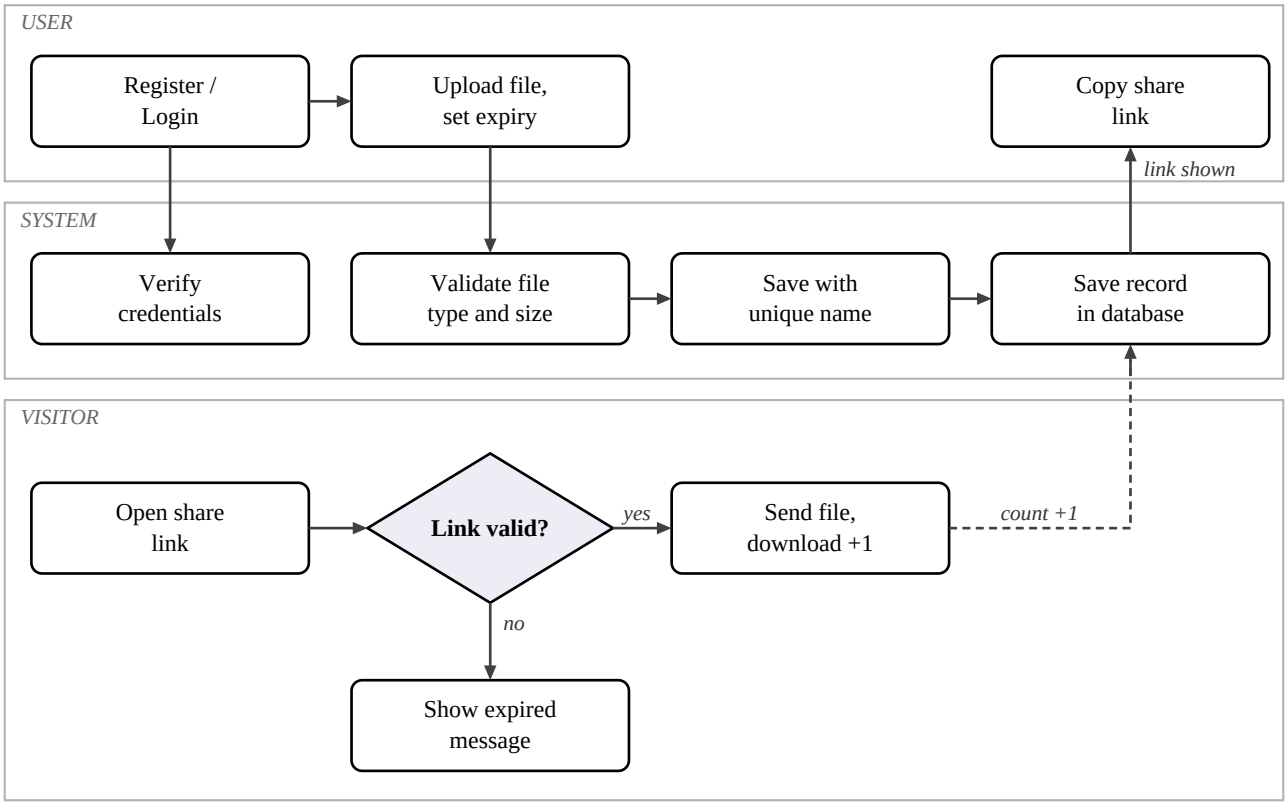


Figure 4.2 Data flow of upload, share and download in CloudVault

When a visitor opens the share link, the backend looks up the token in the database and compares the current time with the stored expiry date. If the link is still valid, the file is sent to the visitor and the download count is increased by one; otherwise an expiry message is shown. In this way every access to a shared file is checked individually, and an expired link can never deliver a file.

4.3 Database Schema

The database consists of two tables, User and File, with a one-to-many relationship: one user can upload many files, while every file belongs to exactly one user. The relationship is shown in Figure 4.3.

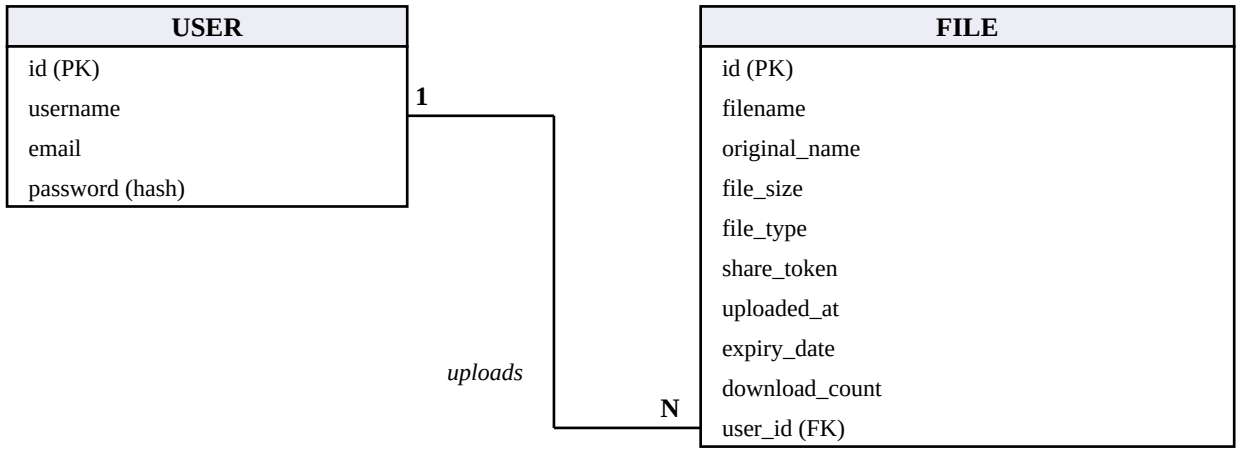


Figure 4.3 Entity relationship diagram of the CloudVault database

The **User** table stores the account details. Its id is an auto-incrementing primary key, the username and email are unique strings, and the password column holds the hashed password produced by Werkzeug. The email address is used as the login name.

The **File** table stores one row for every uploaded file. It keeps the unique stored file name (UUID prefix plus the sanitized name), the original name as uploaded by the user, the file size in bytes and the file extension. The share_token column holds the unique UUID used in the share link, uploaded_at records the upload time in UTC, expiry_date records when the link stops working, and download_count records how many times the file has been downloaded. The user_id column is a foreign key that links each file to its owner.

5. CLOUDVAULT SYSTEM

5.1 Existing System

Files are commonly shared today through email attachments, messaging applications and commercial cloud services. Each of these methods works in daily life, but each also has limitations when the owner wants controlled and temporary sharing.

- File sharing through email attachments is limited by size, and the links or copies that are sent never expire automatically.
- Commercial platforms such as Google Drive and Dropbox require third-party accounts and have storage caps.
- Most free-tier tools do not provide per-file download tracking.
- There is no fine-grained access control for individual file sharing links.

- There is no lightweight, self-hosted option that is suitable for academic internship projects and small teams.

Because of these drawbacks, a user who only wants to hand over a document for a few days must either accept permanent exposure of the file or manage access manually. A simple system that combines authenticated storage, expiring links and download tracking therefore fills a real need.

5.2 Proposed System

The proposed CloudVault is a full-stack web application that digitizes and centralizes secure file storage and sharing. It is built with Python Flask, SQLite and Bootstrap 5, and it offers the following features.

- **User-authenticated access** – only registered users can upload and manage files.
- **UUID-based unique shareable links** – every file receives its own link, which recipients can use without logging in.
- **Configurable link expiry** – 1, 3, 7 or 30 days, validated automatically on every access.
- **Real-time download tracking** – the download count of each file is stored persistently in the database.
- **Dashboard with live statistics** – total files, total downloads and active links.
- **Complete file deletion** – removes both the physical file from the server and its database record.
- **Responsive interface** – a Bootstrap 5 layout that works on mobile, tablet and desktop screens.

By combining these features, CloudVault gives the owner full control over who may download a file and for how long, while keeping the application simple enough to run on an ordinary computer without any external service.

5.3 System Capabilities

The implemented system provides the following capabilities, all of which were developed and tested during the internship.

- **User Registration** – creates a new account with username, email and hashed password.
- **User Login and Logout** – session-based login with @login_required protection for private pages.
- **File Upload** – type-validated upload with a 16 MB limit and UUID-named file storage.

- **Shareable Link** – a unique UUID token and URL for every file.
- **File Expiry** – automatic expiry after 1, 3, 7 or 30 days with access validation.
- **Download Count** – increases automatically on every valid download.
- **File Delete** – removes the physical file and the database record.
- **Dashboard Statistics** – shows total files, downloads and active links.
- **File Search** – real-time JavaScript search by file name.
- **Drag and Drop** – drop-to-upload with a live file name preview.
- **Responsive Interface** – Bootstrap 5 layout for all screen sizes.

6. PROPOSED METHOD

6.1 User Authentication

Registration collects the username, email and password. Werkzeug's generate_password_hash() hashes the password using PBKDF2-SHA256 before it is stored in the User table in SQLite, so passwords are never kept as plain text. During login, the submitted password is verified against the stored hash with check_password_hash(). Flask-Login then creates and manages a secure session cookie. The @login_required decorator protects the upload, download, dashboard and file management routes, and redirects unauthenticated requests to the login page automatically.

Duplicate accounts are avoided by checking whether the email address already exists before a new user record is created. Error and success messages are shown to the user through Flask's flash mechanism, so the user always knows the result of an action.

6.2 File Upload Module

The /upload route accepts an HTTP POST request with a multipart form containing the file and the expiry_days value. The file extension is checked against ALLOWED_EXTENSIONS, which contains pdf, jpg, jpeg, png, gif, doc, docx, txt and zip. Flask's MAX_CONTENT_LENGTH setting enforces the 16 MB size limit and returns a 413 error for oversized files. Werkzeug's secure_filename() sanitizes the original name to prevent directory traversal attacks, and a UUID4 prefix is added so that the stored name is unique across all users.

The physical file is saved in the uploads folder, and its metadata is inserted into the File table through SQLAlchemy session operations. Because the stored name is unique, two users can

upload files with the same original name without overwriting each other, and the original name is kept separately so that it can be restored when the file is downloaded.

6.3 Share Link Generation

At upload time, every file is assigned a unique share_token generated with Python's uuid.uuid4() function. The token is stored in the File table and forms the shareable URL /share/<token>. When the link is opened, the Flask route queries the File table for the matching token. If the file is found and has not expired, it is served with send_from_directory() using the original file name as the download name.

The download_count field is increased by one and committed to the database on every valid access. This gives the owner a precise record of how often each file has been downloaded through its link.

6.4 File Expiry Control

At upload, the user chooses an expiry period from a dropdown of 1 day, 3 days, 7 days or 30 days. The expiry_date is calculated as the current UTC time plus the chosen number of days and is stored in the File table. When a share link is opened, Flask evaluates whether the current UTC time is later than the stored expiry_date. If it is, access is denied and the flash message "This link has expired!" is shown.

The My Files page reads the expiry_date of every file to display a green Active badge or a red Expired badge beside it, so the owner can see at a glance which links are still working.

6.5 File Download and Delete

A logged-in user can download a file through the /download/<id> route, which serves the file with send_from_directory() and increments the download_count. For deletion, the /delete/<id> route removes the physical file from the uploads folder using os.remove(), after checking that the file exists to avoid errors, and then deletes the File record with SQLAlchemy's db.session.delete() followed by db.session.commit(). A JavaScript confirm() dialog in the browser prevents accidental permanent deletion.

6.6 Dashboard and File Management

The dashboard is the home page of a logged-in user. It shows a welcome banner, three statistic cards for total files, total downloads and active links, the upload panel with the expiry dropdown, and the five most recent uploads with download, copy link and delete buttons. The My Files page lists every file of the user with its size, upload time, type badge, status badge, download count and expiry date.

Both pages receive their data from the controller as a list of File objects. The statistics are calculated from that list by the Jinja2 template, for example by summing the download counts of all files, so the numbers always reflect the current state of the database.

6.7 Front-End Interaction

The JavaScript file main.js adds several conveniences without reloading the page. The drag-and-drop handler lets the user drop a file on the upload area and shows its name and size. The search function filters the file list while the user types. The Clipboard API copies the share link, and a toast notification confirms the action. Flash alerts are hidden automatically after a few seconds. The purple gradient theme, card layout and responsive breakpoints in style.css give the application a consistent look on all screen sizes.

7. ALGORITHM

7.1 User Registration – Algorithm

1. Accept the username, email and password from the registration form.
2. Query the User table to check whether the email is already registered.
3. If the user already exists, flash "Email already registered!", redirect to /register and stop.
4. Convert the password into a hash using generate_password_hash().
5. Create a new User object with the username, email and hashed password.
6. Add the object to the database session and commit it.
7. Flash "Registration successful!" and redirect to /login.

7.2 User Login – Algorithm

1. Accept the email and password from the login form.
2. Search the User table for the entered email.
3. If no user is found or check_password_hash() fails, flash "Invalid credentials!" and redirect to /login.
4. If the credentials are valid, call login_user() so that Flask-Login creates the session cookie.
5. Redirect the user to /dashboard.

7.3 File Upload – Algorithm

1. Check whether the key 'file' exists in request.files; if not, flash an error and stop.

2. Read expiry_days from the form (default 7).
3. Validate the file extension against ALLOWED_EXTENSIONS; if it is not allowed, flash "File type not allowed!" and stop.
4. Sanitize the file name with secure_filename().
5. Create the unique name by adding a UUID4 prefix to the sanitized name.
6. Build the file path inside the uploads folder and save the file.
7. Calculate expiry_date as the current UTC time plus expiry_days.
8. Generate a unique share_token with uuid.uuid4().
9. Create the File object with all metadata, add it to the session and commit.
10. Flash "File uploaded successfully!" and redirect to /dashboard.

7.4 Share Link Access – Algorithm

1. Extract the token from the URL /share/<token>.
2. Find the file with that share_token; if none exists, return a 404 page.
3. If expiry_date is set and the current UTC time is later than it, flash "This link has expired!", redirect to the home page and stop.
4. Increase download_count by one and commit the change.
5. Send the file as an attachment using its original name.

7.5 File Expiry Check – Algorithm

1. Retrieve all files that belong to the current user.
2. For each file, if expiry_date is empty, label it "No Expiry".
3. Otherwise, if the current UTC time is not later than expiry_date, label it "Active"; if it is later, label it "Expired".
4. Pass the label to the Jinja2 template, which displays the green or red badge.
5. In the /share route, block access whenever the file is expired.

7.6 File Delete – Algorithm

1. Accept the file_id from the URL /delete/<file_id>.
2. Fetch the file record; if it does not exist, return a 404 page.
3. Build the path of the stored file inside the uploads folder.

4. If the file exists on disk, remove it with os.remove().
5. Delete the file record from the database and commit.
6. Flash "File deleted!" and redirect to /dashboard.

7.7 Dashboard Statistics – Algorithm

1. Retrieve the files of the current user, ordered by upload time with the newest first.
2. Pass the list to the dashboard template.
3. In the template, count the files to obtain the total number of files.
4. Add the download_count of every file to obtain the total downloads.
5. Count the files that have an expiry date to obtain the active link figure.
6. Display the statistic cards and the five most recent files.

8. CODE IMPLEMENTATION

8.1 Project File Structure

The project follows a simple and modular layout in which each file has one clear responsibility.

```
cloudvault/
|-- app.py            Main Flask app with all route definitions
|-- models.py         SQLAlchemy User and File database models
|-- config.py         App configuration: keys, limits, extensions
|-- templates/
|   |-- index.html      Landing page with hero and feature cards
|   |-- register.html   User registration form page
|   |-- login.html      User login form page
|   |-- dashboard.html  Statistics dashboard and upload form
|   |-- files.html      Full file list with search and actions
|-- static/
|   |-- css/style.css   Purple gradient theme, hover effects
|   |-- js/main.js      Drag-drop, search, toast, clipboard
|-- uploads/          Physical file storage (created on first run)
|-- instance/
    |-- database.db   SQLite database (created on first run)
```

8.2 config.py – Application Configuration

Stores the secret key, the database location, the upload folder, the maximum upload size and the allowed file types.

```
import os

class Config:
    SECRET_KEY = 'your_secret_key_here'
    SQLALCHEMY_DATABASE_URI = 'sqlite:///database.db'
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    UPLOAD_FOLDER = 'uploads'
    MAX_CONTENT_LENGTH = 16 * 1024 * 1024  # 16MB max
    ALLOWED_EXTENSIONS = {'png', 'jpg', 'jpeg', 'gif', 'pdf', 'doc', 'docx', 'txt', 'zip'}
```

8.3 models.py – Database Models

Defines the User and File tables and the one-to-many relationship between them.

```
from flask_sqlalchemy import SQLAlchemy
from flask_login import UserMixin
from datetime import datetime

db = SQLAlchemy()

class User(UserMixin, db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(80), unique=True, nullable=False)
    email = db.Column(db.String(120), unique=True, nullable=False)
    password = db.Column(db.String(200), nullable=False)
    files = db.relationship('File', backref='owner', lazy=True)

class File(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    filename = db.Column(db.String(200), nullable=False)
    original_name = db.Column(db.String(200), nullable=False)
    file_size = db.Column(db.Integer)
    file_type = db.Column(db.String(50))
    share_token = db.Column(db.String(100), unique=True)
    uploaded_at = db.Column(db.DateTime, default=datetime.utcnow)
    expiry_date = db.Column(db.DateTime, nullable=True)
    download_count = db.Column(db.Integer, default=0)
    user_id = db.Column(db.Integer, db.ForeignKey('user.id'), nullable=False)
```

8.4 app.py – Application Setup

Creates the Flask application, connects the database, sets up Flask-Login and defines the file type check.

```
from flask import Flask, render_template, redirect, url_for, request, flash, send_from_directory
from flask_login import LoginManager, login_user, logout_user, login_required, current_user
from werkzeug.security import generate_password_hash, check_password_hash
from werkzeug.utils import secure_filename
from datetime import datetime, timedelta
from models import db, User, File
from config import Config
import os, uuid

app = Flask(__name__)
app.config.from_object(Config)

db.init_app(app)

login_manager = LoginManager()
login_manager.init_app(app)
login_manager.login_view = 'login'

@login_manager.user_loader
def load_user(user_id):
    return User.query.get(int(user_id))

def allowed_file(filename):
    return '.' in filename and \
           filename.rsplit('.', 1)[1].lower() in app.config['ALLOWED_EXTENSIONS']
```

8.5 Registration Module Code

Handles new user account creation, password hashing and duplicate email validation.

```
@app.route('/register', methods=['GET', 'POST'])
def register():
```

```
    if request.method == 'POST':
        username = request.form['username']
        email = request.form['email']
        password = generate_password_hash(request.form['password'])

        existing_user = User.query.filter_by(email=email).first()
        if existing_user:
            flash('Email already registered!', 'danger')
            return redirect(url_for('register'))

        new_user = User(username=username, email=email, password=password)
        db.session.add(new_user)
        db.session.commit()
        flash('Registration successful! Please login.', 'success')
        return redirect(url_for('login'))
    return render_template('register.html')
```

8.6 Login and Logout Module Code

Contains the authentication logic that validates user credentials and ends the session.

```
@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        email = request.form['email']
        password = request.form['password']
        user = User.query.filter_by(email=email).first()
        if user and check_password_hash(user.password, password):
            login_user(user)
            return redirect(url_for('dashboard'))
        flash('Invalid credentials!', 'danger')
    return render_template('login.html')

@app.route('/logout')
@login_required
def logout():
    logout_user()
    return redirect(url_for('index'))
```

8.7 File Upload Module Code

Validates the file, stores it with a unique name and saves its details together with the share token and expiry date.

```
@app.route('/upload', methods=['POST'])
@login_required
def upload():
    if 'file' not in request.files:
        flash('No file selected!', 'danger')
        return redirect(url_for('dashboard'))

    file = request.files['file']
    expiry_days = int(request.form.get('expiry_days', 7))

    if file and allowed_file(file.filename):
        original_name = file.filename
        filename = secure_filename(file.filename)
        unique_name = str(uuid.uuid4()) + '_' + filename
        filepath = os.path.join(app.config['UPLOAD_FOLDER'], unique_name)
        file.save(filepath)

        new_file = File(
            filename=unique_name,
            original_name=original_name,
            file_size=os.path.getsize(filepath),
            file_type=filename.rsplit('.', 1)[1].lower(),
            share_token=str(uuid.uuid4()),
```

```
            expiry_date=datetime.utcnow() + timedelta(days=expiry_days),
            user_id=current_user.id
        )
        db.session.add(new_file)
        db.session.commit()
        flash('File uploaded successfully!', 'success')
    else:
        flash('File type not allowed!', 'danger')
    return redirect(url_for('dashboard'))
```

8.8 Download and Share Link Module Code

The download route serves a file to its logged-in user, and the share route serves a file to anyone who holds a valid, unexpired link. Both increase the download counter.

```
@app.route('/download/<int:file_id>')
@login_required
def download(file_id):
    file = File.query.get_or_404(file_id)
    file.download_count += 1
    db.session.commit()
    return send_from_directory(app.config['UPLOAD_FOLDER'], file.filename,
                               as_attachment=True, download_name=file.original_name)

@app.route('/share/<token>')
def share(token):
    file = File.query.filter_by(share_token=token).first_or_404()
    if file.expiry_date and datetime.utcnow() > file.expiry_date:
        flash('This link has expired!', 'danger')
        return redirect(url_for('index'))
    file.download_count += 1
    db.session.commit()
    return send_from_directory(app.config['UPLOAD_FOLDER'], file.filename,
                               as_attachment=True, download_name=file.original_name)
```

8.9 Dashboard, File Listing and Deletion Code

The dashboard and file list routes show the files of the user, and the delete route removes a file from the disk and from the database.

```
@app.route('/dashboard')
@login_required
def dashboard():
    files = File.query.filter_by(user_id=current_user.id).order_by(File.uploaded_at.desc()).all()
    return render_template('dashboard.html', files=files)

@app.route('/files')
@login_required
def files():
    all_files = File.query.filter_by(user_id=current_user.id).all()
    return render_template('files.html', files=all_files, now=datetime.utcnow())

@app.route('/delete/<int:file_id>')
@login_required
def delete(file_id):
    file = File.query.get_or_404(file_id)
    filepath = os.path.join(app.config['UPLOAD_FOLDER'], file.filename)
    if os.path.exists(filepath):
        os.remove(filepath)
    db.session.delete(file)
    db.session.commit()
    flash('File deleted!', 'success')
    return redirect(url_for('dashboard'))

if __name__ == '__main__':
    with app.app_context():
```

```
        os.makedirs('uploads', exist_ok=True)
        db.create_all()
    app.run(debug=True)
```

8.10 Front-End Script Code – main.js

Copies the share link to the clipboard and filters the file list while the user types.

```
function copyLink(link) {
    navigator.clipboard.writeText(link).then(() => {
        showToast('Share link copied to clipboard!');
    }).catch(() => {
        prompt('Copy this link:', link);
    });
}

function searchFiles() {
    const input = document.getElementById('searchInput').value.toLowerCase();
    const cards = document.querySelectorAll('.file-card');
    cards.forEach(card => {
        const name = card.dataset.name || '';
        card.style.display = name.includes(input) ? 'block' : 'none';
    });
}
```

9. SAMPLE OUTPUT

9.1 Landing Page (index.html)

The landing page displays a purple gradient background (#667eea to #764ba2) with the CloudVault title, a one-line description and three Bootstrap feature cards: Secure Upload, Shareable Links and File Expiry. The Register and Login buttons are placed in the top navigation bar.

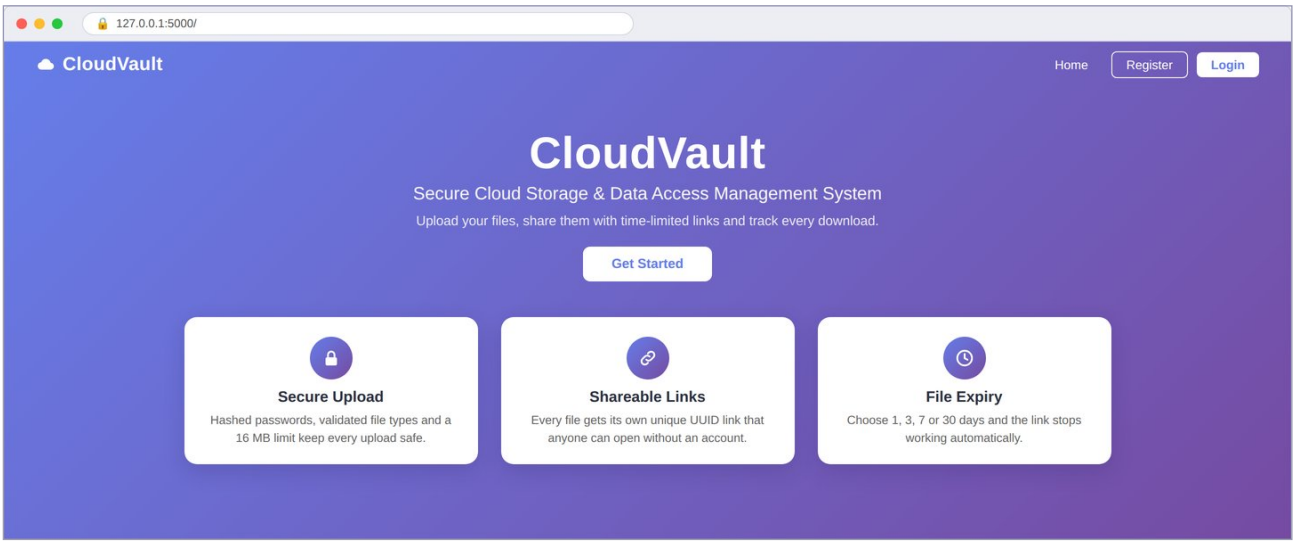


Figure 9.1 Landing page

9.2 Register Page

A centered white card with the heading Create Account, form fields for username, email and password, and a purple Register button. Flash messages appear in a red alert box for errors such as "Email already registered!".

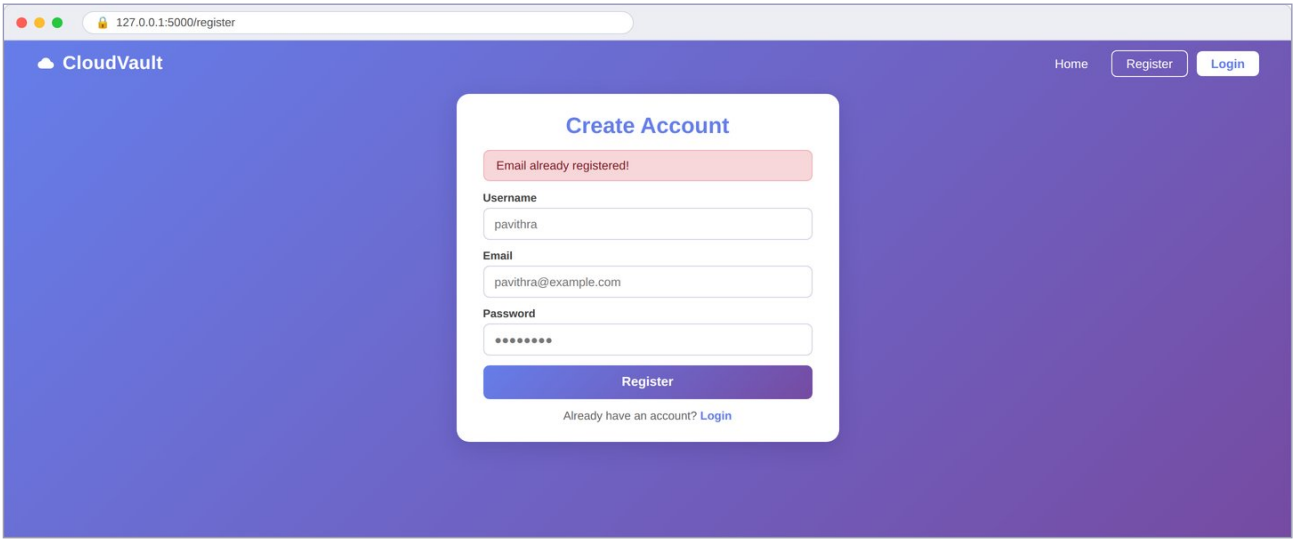


Figure 9.2 Register page

9.3 Login Page

A centered card with the heading Welcome Back, email and password fields and a Login button. On invalid credentials, a red flash message "Invalid credentials!" is displayed. A successful login redirects the user to the dashboard.

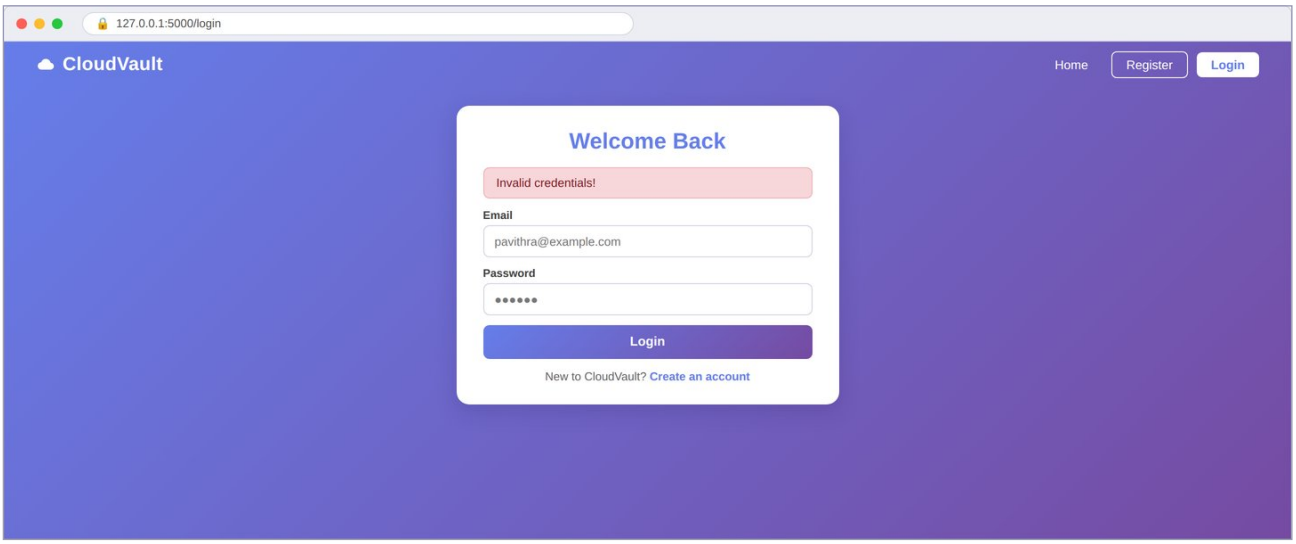


Figure 9.3 Login page

9.4 Dashboard Page

Three Bootstrap statistic cards at the top show the total files uploaded, the total downloads across all files and the number of active links. Below them is a drag-and-drop upload form with an expiry dropdown. The right section shows the five most recent uploads with Download, Copy Link and Delete buttons.

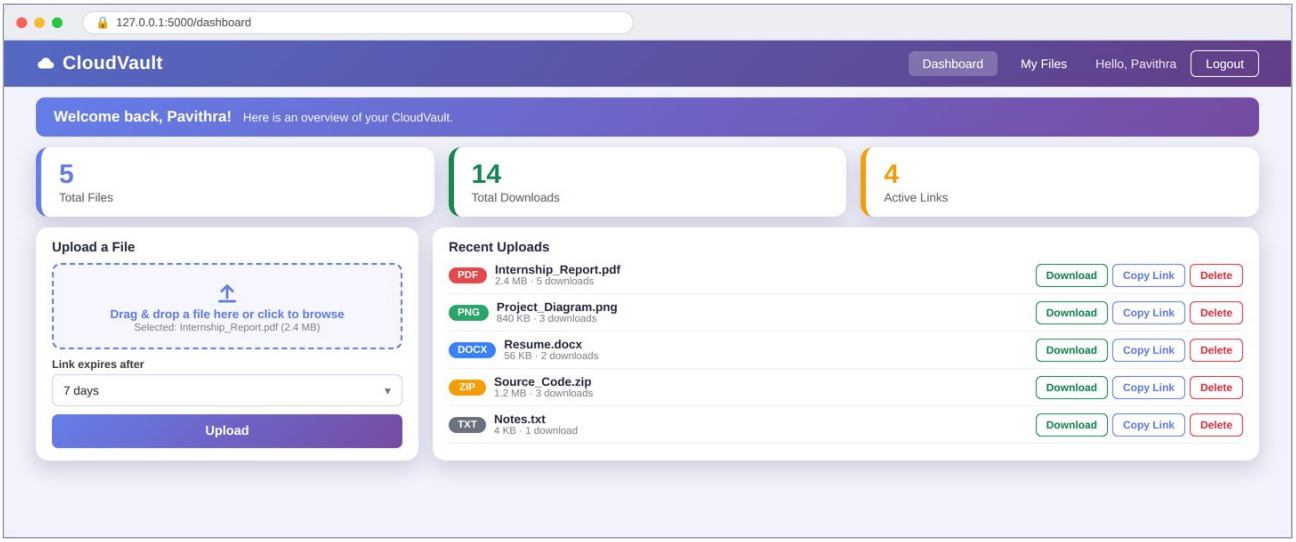


Figure 9.4 Dashboard page

9.5 Files Page (files.html)

A full list of all uploaded files with the original file name, a file type badge, a green Active or red Expired status badge, the download count, and the Copy Link, Download and Delete buttons. A real-time search bar filters the files by name as the user types, without any page reload.

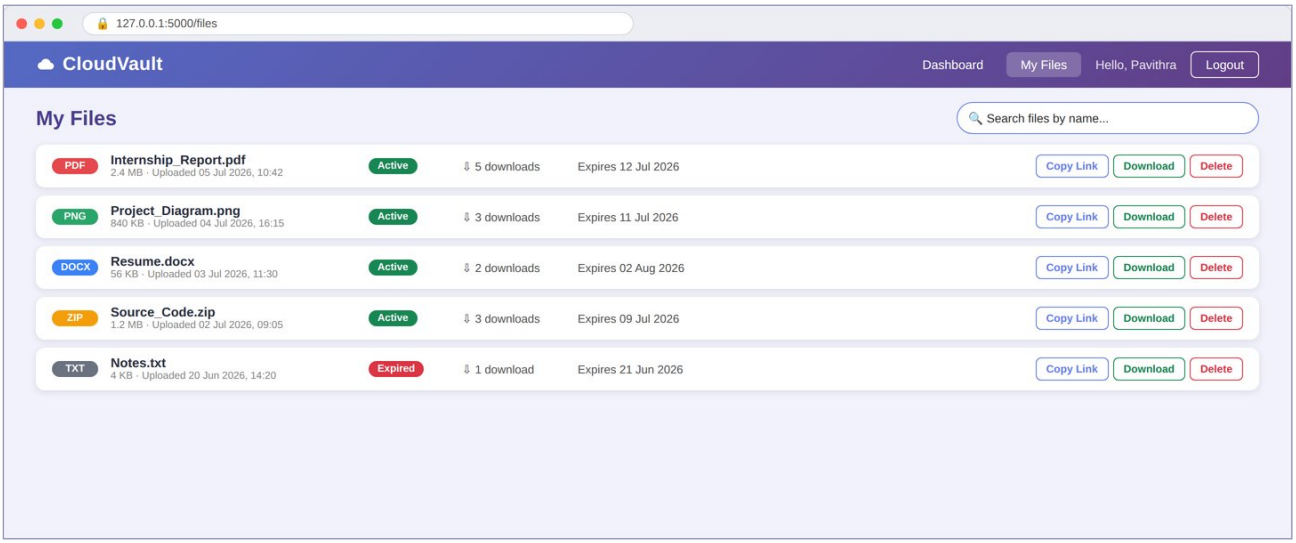


Figure 9.5 Files page

9.6 Share Link Access Output

When a valid, unexpired share link is opened by anyone, with or without an account, the file is served as a browser download under its original name and the download count in the database increases. When an expired link is opened, the user is redirected to the landing page and the message "This link has expired!" is displayed in a red alert.

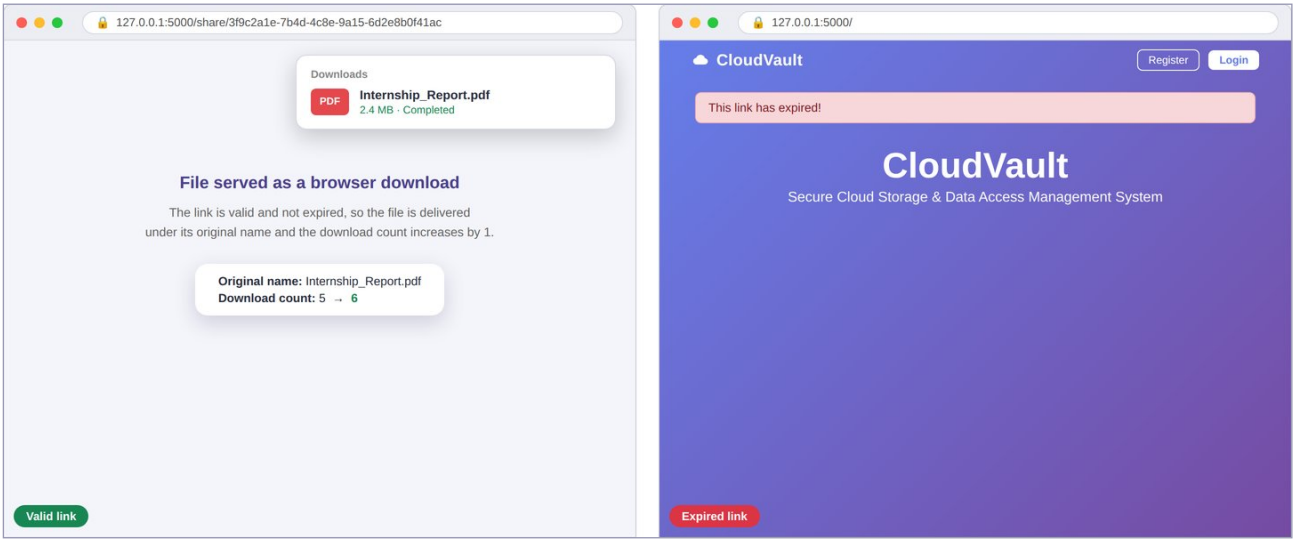


Figure 9.6 Share link access output

10. ANALYSIS

Testing of CloudVault was carried out module by module to verify both the functional correctness of each feature and the behavior of the system as a whole. Fourteen test cases were designed to cover normal use as well as edge cases, and every one of them produced the expected result.

Authentication testing. Registration with a new, valid email created the account and redirected the user to the login page, while registration with an email that already existed was rejected with the message "Email already registered!". Login with correct credentials created a session and opened the dashboard, and login with a wrong password produced the message "Invalid credentials!" without creating a session.

File upload testing. A PDF or JPG file within the size limit was saved and its database record was created. A file with a type that is not allowed, such as an .exe file, was rejected with the message "File type not allowed!", and a file larger than 16 MB was refused by Flask with a 413 Request Entity Too Large response.

Share link and expiry testing. Opening a valid share link downloaded the file under its original name and increased the download count by one. Opening a link whose expiry date had passed redirected the visitor to the landing page with the message "This link has

expired!" and did not deliver the file, which confirms that expiry is enforced on the server and not only displayed in the interface.

Dashboard and interface testing. After several uploads, the dashboard displayed the correct totals for files, downloads and active links. The delete button removed both the physical file and the database record. The real-time search filtered the list instantly while typing, drag and drop showed a file name preview, and the Bootstrap layout adjusted correctly between mobile and desktop screen widths.

Performance. Pages and file operations responded quickly during testing because of the lightweight Flask server and the small SQLite database. For a larger number of users and files, a server database such as MySQL or PostgreSQL and cloud object storage would be required.

Usability. The dashboard reduces the work of sharing a file to a few steps: select the file, choose the expiry period and copy the link. The Active and Expired badges and the download counts make the state of every shared file visible without any extra action.

Security observations. Passwords are stored as salted hashes, private pages require login, uploaded names are sanitized and made unique, only whitelisted file types are accepted, and share tokens are random UUIDs that are practically impossible to guess. The analysis also identified areas that can be strengthened in a production version, namely an ownership check on the download and delete routes, CSRF protection for state-changing requests, moving the secret key into an environment variable, and serving the application over HTTPS. These are included in the future work.

Comparative analysis. Compared with email attachments and permanent public links, CloudVault lets the owner decide how long a link stays valid, shows how many times a file has been downloaded, and keeps all files in one place. Overall, the analysis confirms that the system meets its objectives of secure file storage, controlled and time-limited sharing, and convenient file management.

11. CONCLUSION

The **CloudVault — Secure Cloud Storage and Data Access Management System** was successfully designed, developed and tested during the one-month offline Summer Internship at DLK Software Solutions, Vadapalani, Chennai, from 06 June 2026 to 06 July 2026, under the mentorship of Mr. Ram and the guidance of Faculty Incharge Ms. Ambika.

Starting with no prior knowledge of Python Flask, a complete and fully functional cloud storage web application was built within the four-week internship period. The system covers all the planned features: user authentication with hashed passwords, file upload with validation and UUID-based unique naming, shareable link generation, configurable file expiry, real-time download tracking, a statistics dashboard, file search, drag-and-drop upload and a fully responsive Bootstrap 5 interface. All 14 test cases passed successfully.

The internship gave deep hands-on exposure to Python Flask, the SQLAlchemy ORM, Flask-Login, Werkzeug, Jinja2 templating, Bootstrap 5 and JavaScript. It complemented the author's existing MERN stack skills and showed that a new technology stack can be learned quickly in a professional environment.

CloudVault is a practical web application suitable for secure document sharing in academic settings, temporary file sharing between small teams, and personal file backup. The project shows that Python Flask can deliver clean, secure and maintainable web applications with minimal setup.

12. FUTURE WORK

The following enhancements are identified for future versions of CloudVault.

- **Access control hardening:** Verify that the logged-in user owns a file before it is downloaded or deleted, use POST requests with CSRF tokens for delete actions, and keep the secret key in an environment variable.
- **Cloud deployment:** Deploy the application on AWS EC2 or Render.com with AWS S3 integration for scalable and persistent file storage beyond the local file system.
- **Email notifications:** Use Python's smtplib to notify recipients by email automatically when a file link is shared with them.
- **Password-protected links:** Add an optional PIN or password to shareable links so that recipients must enter a code before downloading sensitive files.
- **In-browser file preview:** Preview PDF documents and images using PDF.js so that recipients can view files without downloading them.
- **Storage quota per user:** Enforce per-user upload limits in the database to control the total storage used and prevent misuse.
- **Admin dashboard:** Build a separate admin panel to view all registered users, uploaded files and download activity, and to manage system-wide settings.

- **REST API endpoints:** Expose RESTful endpoints for programmatic upload, download and file management so that other applications can integrate with CloudVault.
- **File versioning:** Store multiple versions of an updated file and allow users to restore a previous version.
- **Folder organization:** Allow users to create named folders and organize their files into categories.
- **Mobile application:** Develop a React Native or Flutter app for file access and upload directly from smartphones.

13. REFERENCES

1. Flask Documentation, Pallets Projects — https://flask.palletsprojects.com/
2. SQLAlchemy Documentation — https://docs.sqlalchemy.org/en/14/
3. Flask-Login Documentation — https://flask-login.readthedocs.io/en/latest/
4. Werkzeug Documentation, Pallets Projects — https://werkzeug.palletsprojects.com/
5. Bootstrap 5 Documentation — https://getbootstrap.com/docs/5.3/
6. Python UUID Module, Python Standard Library — https://docs.python.org/3/library/uuid.html
7. Jinja2 Template Engine — https://jinja.palletsprojects.com/en/3.1.x/
8. SQLite Documentation — https://www.sqlite.org/docs.html
9. Python datetime Module — https://docs.python.org/3/library/datetime.html
10. Werkzeug Security Utilities — https://werkzeug.palletsprojects.com/en/2.3.x/utils/
11. MDN Web Docs — HTML5, CSS3, JavaScript — https://developer.mozilla.org/
12. DLK Software Solutions — https://www.dlksoftwaresolutions.co.in/
13. GitHub — J. Pavithra — https://github.com/Pavithra200620

INTERNSHIP CERTIFICATE

